

Lab Manual

by

Kyle Ponte, Enoch Chan

[Stevens.edu](https://www.stevens.edu)

October 18, 2024

© Kyle Ponte, Enoch Chan
Stevens.edu
ALL RIGHTS RESERVED

Lab Manual

Kyle Ponte, Enoch Chan
Stevens.edu

Table 1: Document Update History

Date	Updates
10/2/2024	DDM: Created lab manual and created a new chapter called Advanced Class Modeling Lab (Chapter 1).
10/17/2024	DDM: Created new chapter called Manual Simulink Lab (Chapter 2).

Table of Contents

1	Advanced Class Modeling Lab	
	– <i>Kyle Ponte, Enoch Chan</i>	1
1.1	Section 3.1 Exercise	1
1.2	Section 3.2 Exercise	2
1.3	Section 3.3 Exercise	3
1.4	Section 3.4 Exercise	3
1.5	Section 3.5 Exercise	6
1.6	Section 3.6 Exercise	9
2	Manual Simulink Lab	
	– <i>Kyle Ponte, Enoch Chan</i>	12
2.1	Exercise 7.1	12
2.2	Exercise 7.2	14

List of Tables

1	Document Update History	iii
---	-----------------------------------	-----

List of Figures

1.1	Revised Diagram of Interactive Editor	1
1.2	Diagram for Graphical Document Editor	2
1.3	Motor Diagram	3
1.4	Updated Code Output for Exercise 3.4	5
1.5	State Diagram for Vending Machine	5
1.6	CRC Cards	6
1.7	Use Case Diagram	7
1.8	Object Diagram and Class Diagram	8
1.9	Use-case Diagram for E-Book Website	9
1.10	Class Diagram for E-Book Website	10
1.11	Sequence Diagram for E-Book Website	11
2.1	Diagram of Exercise 7.1 Model	12
2.2	Simulink Scope Showing Min, Max, and Average	13
2.3	Fastest Time	15
2.4	Fastest Time	15
2.5	Fastest Time	16
2.6	Fastest Time	17

Chapter 1

Advanced Class Modeling Lab

– Kyle Ponte, Enoch Chan

1.1 Section 3.1 Exercise

First, a buffer represents an area where lines and boxes are temporarily stored when cut or copied. Next, a selection represents a section of lines and boxes that have been highlighted by the user. Moreover, a sheet contains lines and boxes. Additionally, a line is symbolic of a graphical line that is a part of the sheet and also connects two boxes, while a box is a square graphical element on the sheet. A line segment is a part of a line, and is defined by two points. Lastly, a point shows the endpoints and other important locations in the diagram where lines or line segments meet.

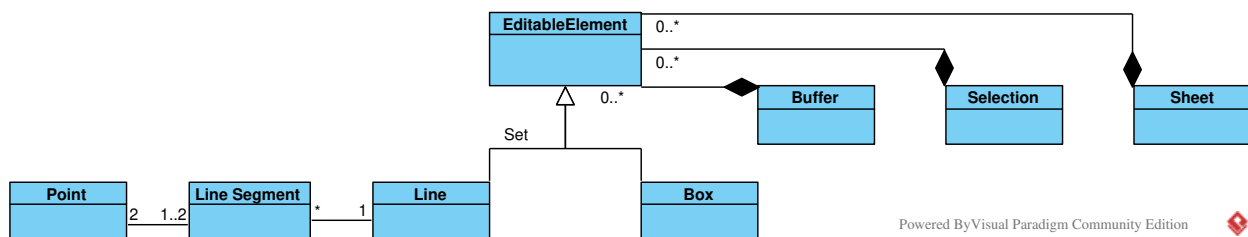


Figure 1.1: Revised Diagram of Interactive Editor

The revised design uses composition to establish clear ownership of **EditableElement** objects (which include **Line** and **Box**). The use of composition ensures that a **Line** or **Box** can only belong to one **Buffer**, **Selection**, or **Sheet** at a time, which prevents issues like shared or conflicting ownership of elements. Introducing the **EditableElement** superclass provides a flexible and scalable structure by applying the generalization concept. This allows common attributes and methods for **Line** and **Box** to be defined once in **EditableElement**, reducing redundancy and improving code reusability. Additionally, managing the association between **EditableElements** and their respective containers (**Buffer**, **Selection**, **Sheet**) becomes easier due to the unified handling of these objects through a single superclass. The revision enforces the constraint that a **Line** or **Box** can belong to exactly one **Buffer**, **Selection**, or **Sheet** through the diagram's structure. This eliminates the possibility of a **Line** or **Box** being in more than one place simultaneously, reducing the risk of data integrity issues and simplifying validation.

1.2 Section 3.2 Exercise

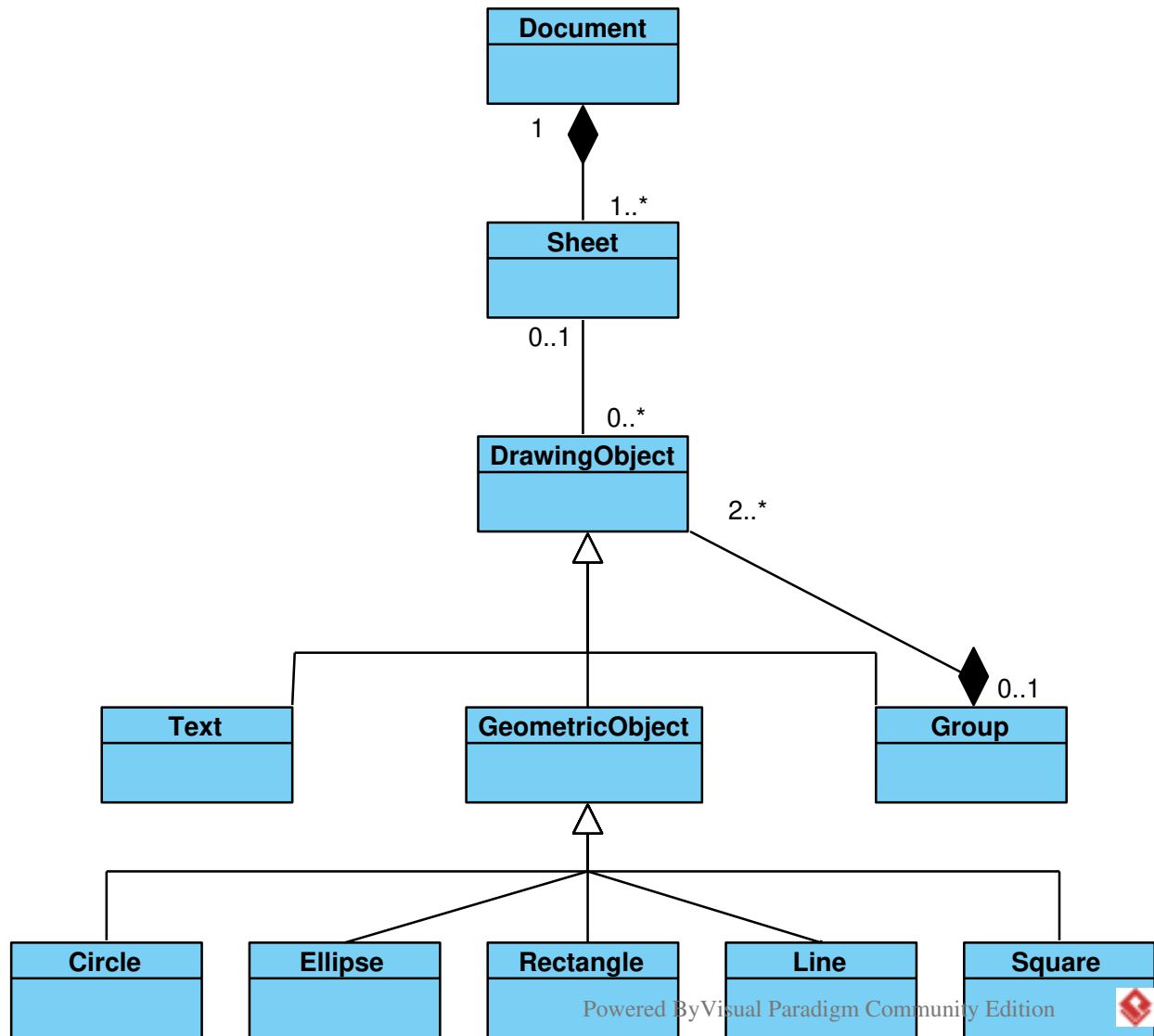


Figure 1.2: Diagram for Graphical Document Editor

1.3 Section 3.3 Exercise

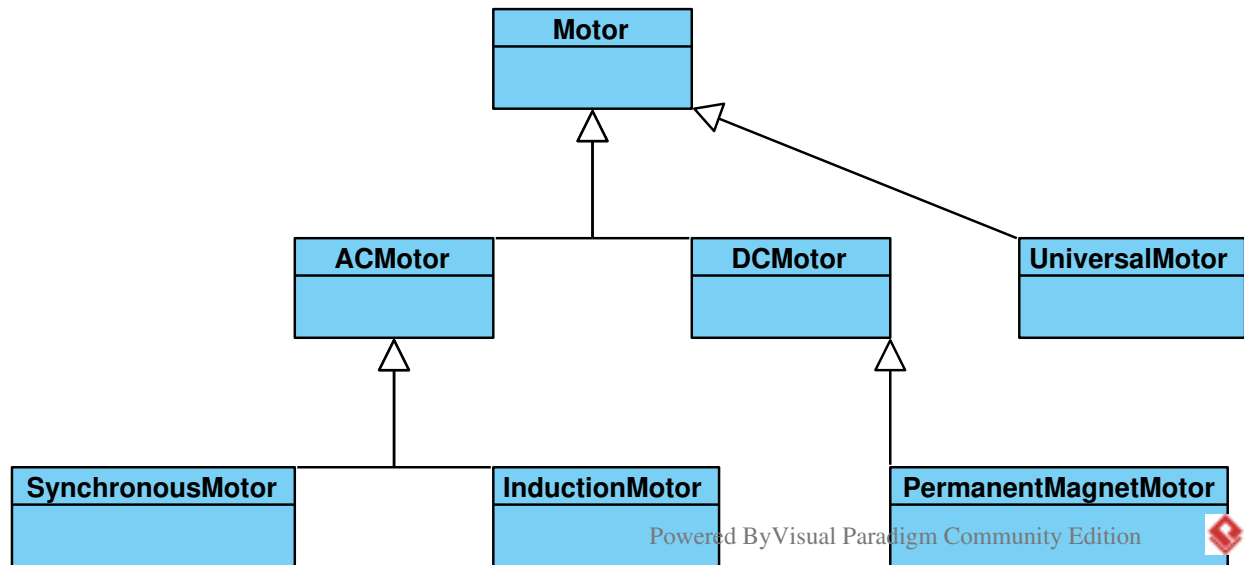


Figure 1.3: Motor Diagram

I did not use multiple inheritance in the initial diagram. This is because ACMotor, DCMotor, and UniversalMotor all inherit from Motor independently. Furthermore, UniversalMotor inherits directly from Motor, not from both ACMotor and DCMotor. This avoids multiple inheritance because UniversalMotor is a standalone subclass of Motor, capable of running on both AC and DC, without needing to inherit from both ACMotor and DCMotor.

1.4 Section 3.4 Exercise

```

1 from abc import ABC
2
3 class SM(ABC):
4     state = 0
5     startState = 0
6
7     def start(self):
8         self.state = self.startState
9
10    # step returns the next output.
11    def step(self, inp):
12        (s, o) = self.getNextValues(self.state, inp)
13        self.state = s
14        return o
15
16    def transduce(self, inputs):
17        self.start()
  
```

```

18         return [self.step(inp) for inp in inputs]
19
20     def run(self, n=10):
21         return self.transduce([None] * n)
22
23     def getNextValues(self, state, inp):
24         nextState = self.getNextState(state, inp)
25         return (nextState, nextState)
26
27     def getNextState(self, state, inp):
28         pass
29
30
31 class VendingMachine(SM):
32     startState = ('waiting', 0) # The machine starts waiting with 0 money
33                                # inserted
34     drink_price = 75 # Price of the drink in cents
35
36     def getNextValues(self, state, inp):
37         current_state, total_money = state
38         if inp == 'cancel':
39             return (('canceled', 0), f"Transaction canceled. Returning ${
40                 total_money / 100:.2f}.")
41         elif inp in [5, 10, 25, 100]: # Valid inputs: nickel, dime, quarter,
42             dollar
43             total_money += inp
44             if total_money >= self.drink_price:
45                 change = total_money - self.drink_price
46                 return (('dispensing', 0), f"Drink dispensed. Returning ${
47                     change / 100:.2f} in change.")
48             else:
49                 return (('waiting', total_money), f"Amount entered: ${
50                     total_money / 100:.2f}. Insert more money.")
51         else:
52             return (state, "Invalid input.")
53
54 # --- Test Scenarios ---
55
56 # Scenario 1: User inputs three quarters
57 vending_machine = VendingMachine()
58 scenario1 = [25, 25, 25] # Three quarters
59 print(vending_machine.transduce(scenario1))
60
61 # Scenario 2: User inputs one quarter and cancels the transaction
62 vending_machine = VendingMachine() # Reset the machine
63 scenario2 = [25, 'cancel'] # One quarter, then cancel
64 print(vending_machine.transduce(scenario2))
65
66 # Scenario 3: User inputs a dime and a dollar bill
67 vending_machine = VendingMachine() # Reset the machine
68 scenario3 = [10, 100] # A dime, then a dollar bill
69 print(vending_machine.transduce(scenario3))

```

```
['Amount entered: $0.25. Insert more money.', 'Amount entered: $0.50. Insert more money.', 'Drink dispensed. Returning $0.00 in change.']  
['Amount entered: $0.25. Insert more money.', 'Transaction canceled. Returning $0.25.']  
['Amount entered: $0.10. Insert more money.', 'Drink dispensed. Returning $0.35 in change.']  
  
** Process exited - Return Code: 0 **  
Press Enter to exit terminal
```

Figure 1.4: Updated Code Output for Exercise 3.4

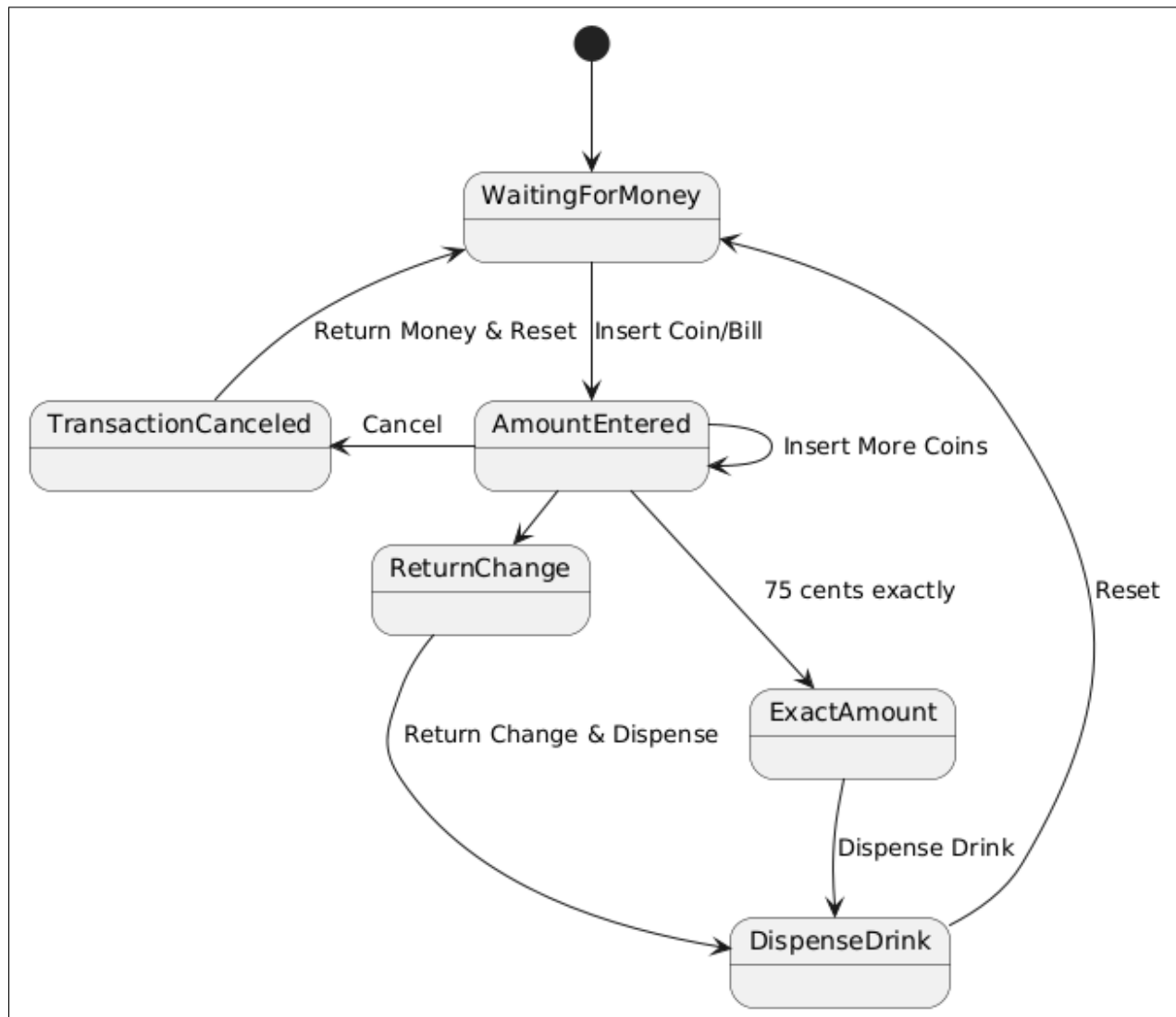


Figure 1.5: State Diagram for Vending Machine

1.5 Section 3.5 Exercise

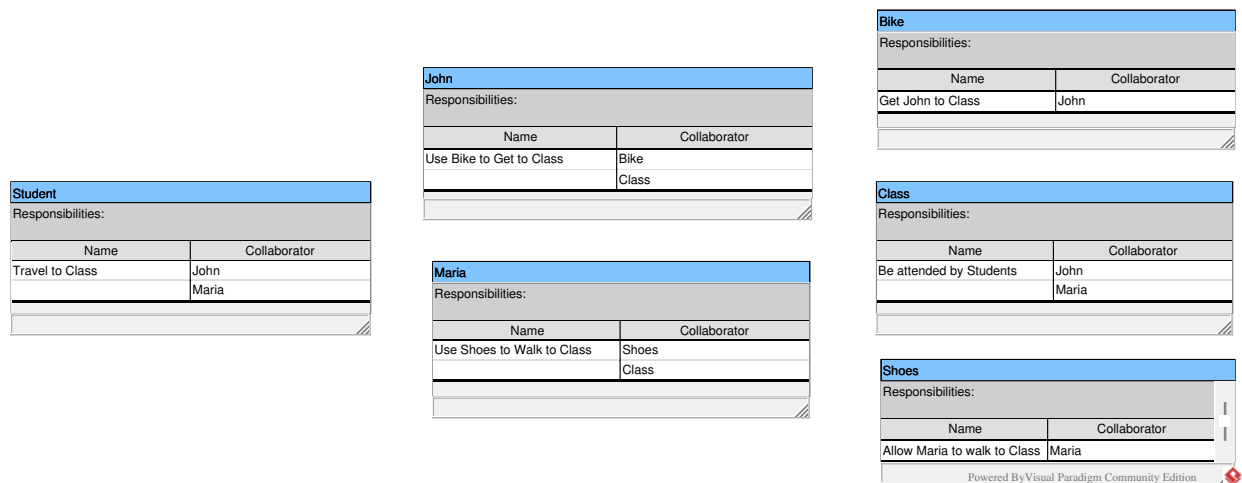


Figure 1.6: CRC Cards

The CRC cards identify the primary classes (Student, Bike, and Shoes) and their responsibilities in the context of students commuting to school. The Student class coordinates interactions with Bike or Shoes, with each class’s card specifying its role in these interactions. This breakdown helps outline responsibilities and collaborations effectively.

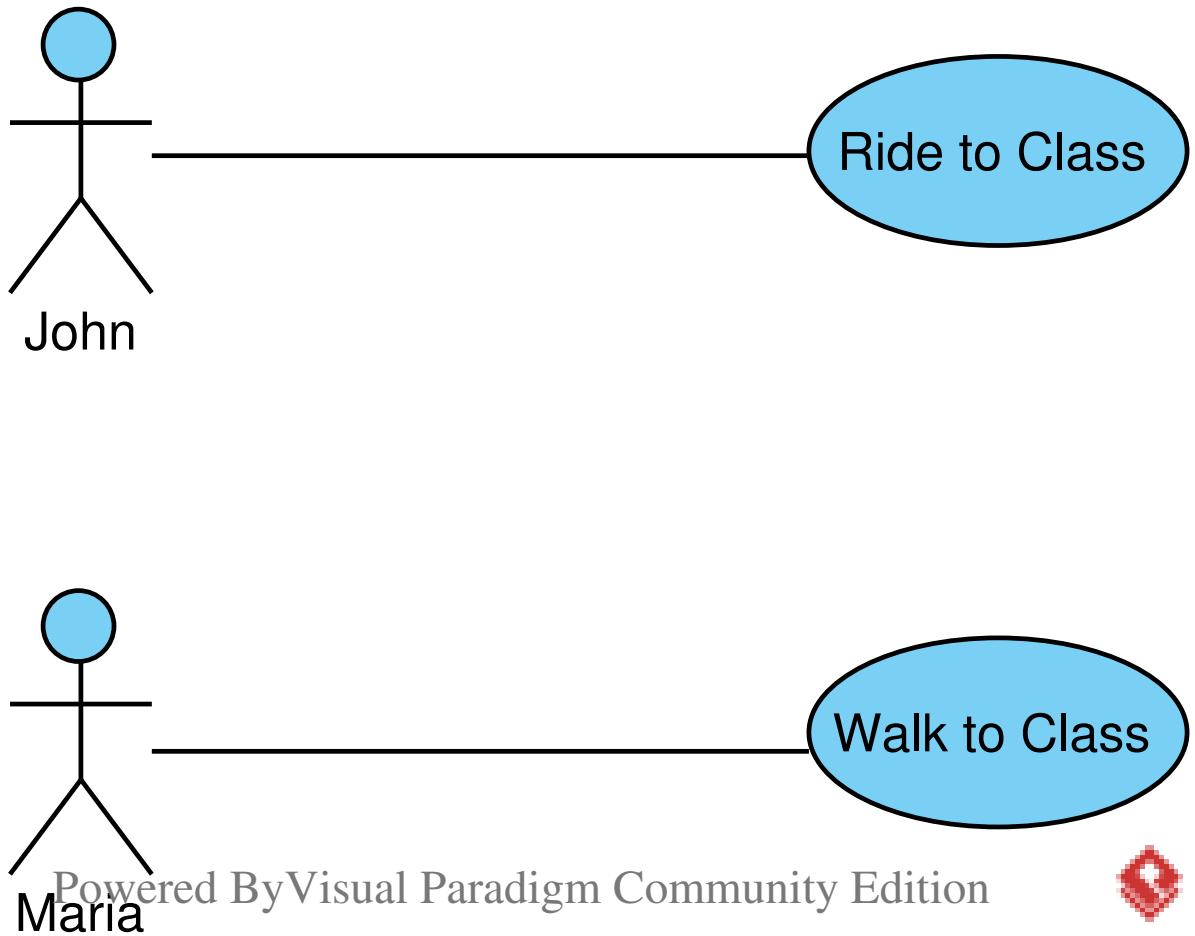
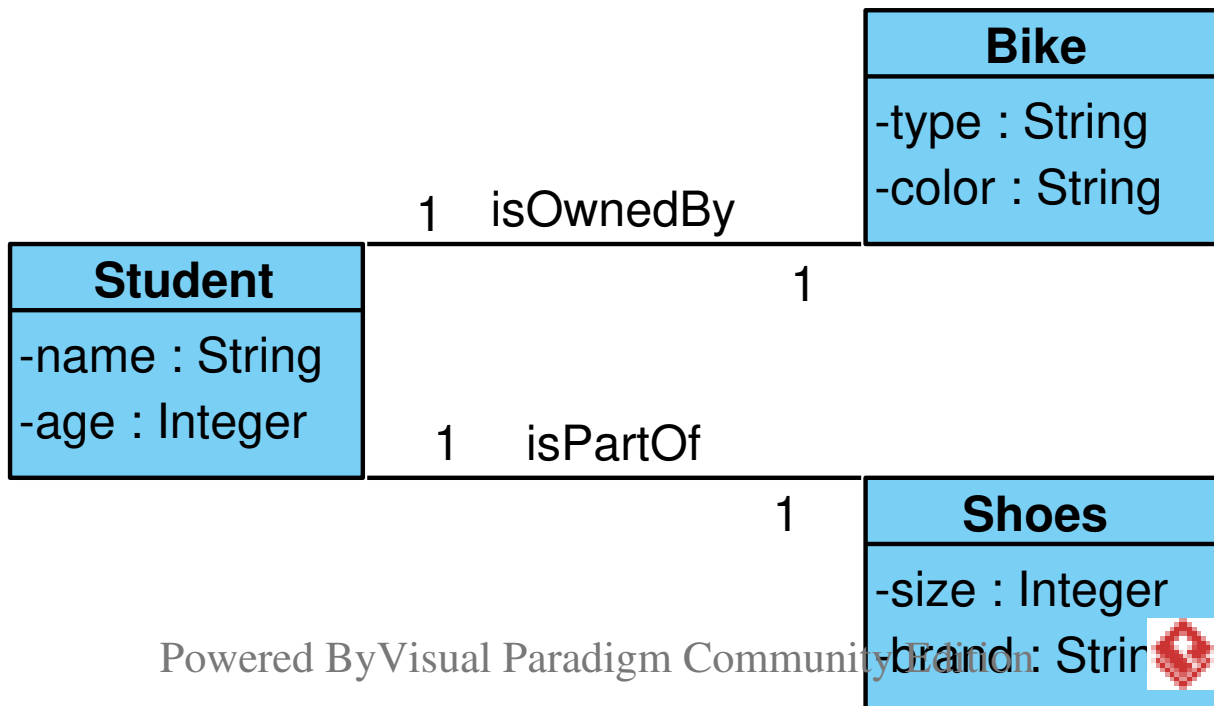
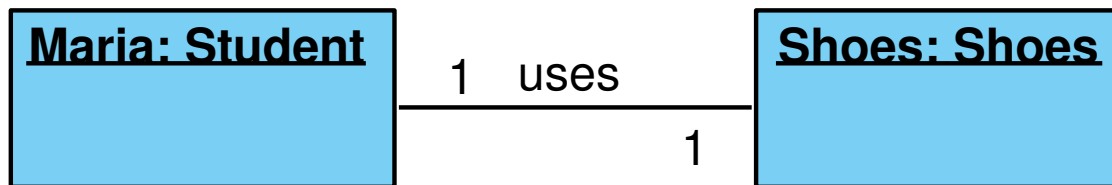
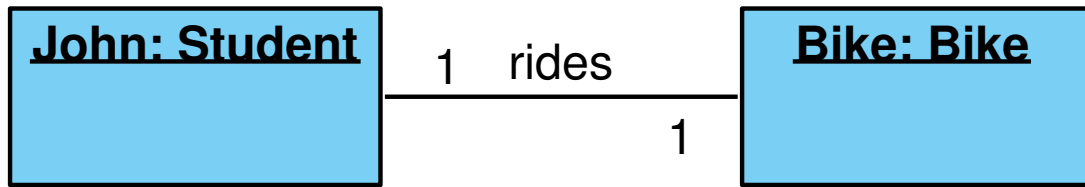


Figure 1.7: Use Case Diagram

The use case diagram depicts the different paths John and Maria take to get to class: John rides a bike, while Maria uses shoes to walk. This diagram visually separates each student's use case, showing how different resources can achieve the same goal of commuting to school.



Powered By Visual Paradigm Community Edition

Figure 1.8: Object Diagram and Class Diagram

The class diagram shows the structural relationships between Student, Bike, and Shoes. The association between Student and Bike is modeled as aggregation, since a bike can exist independently but is linked to a student for commuting. The relationship between Student and Shoes is a composition, as shoes are integral to the student's equipment.

1.6 Section 3.6 Exercise

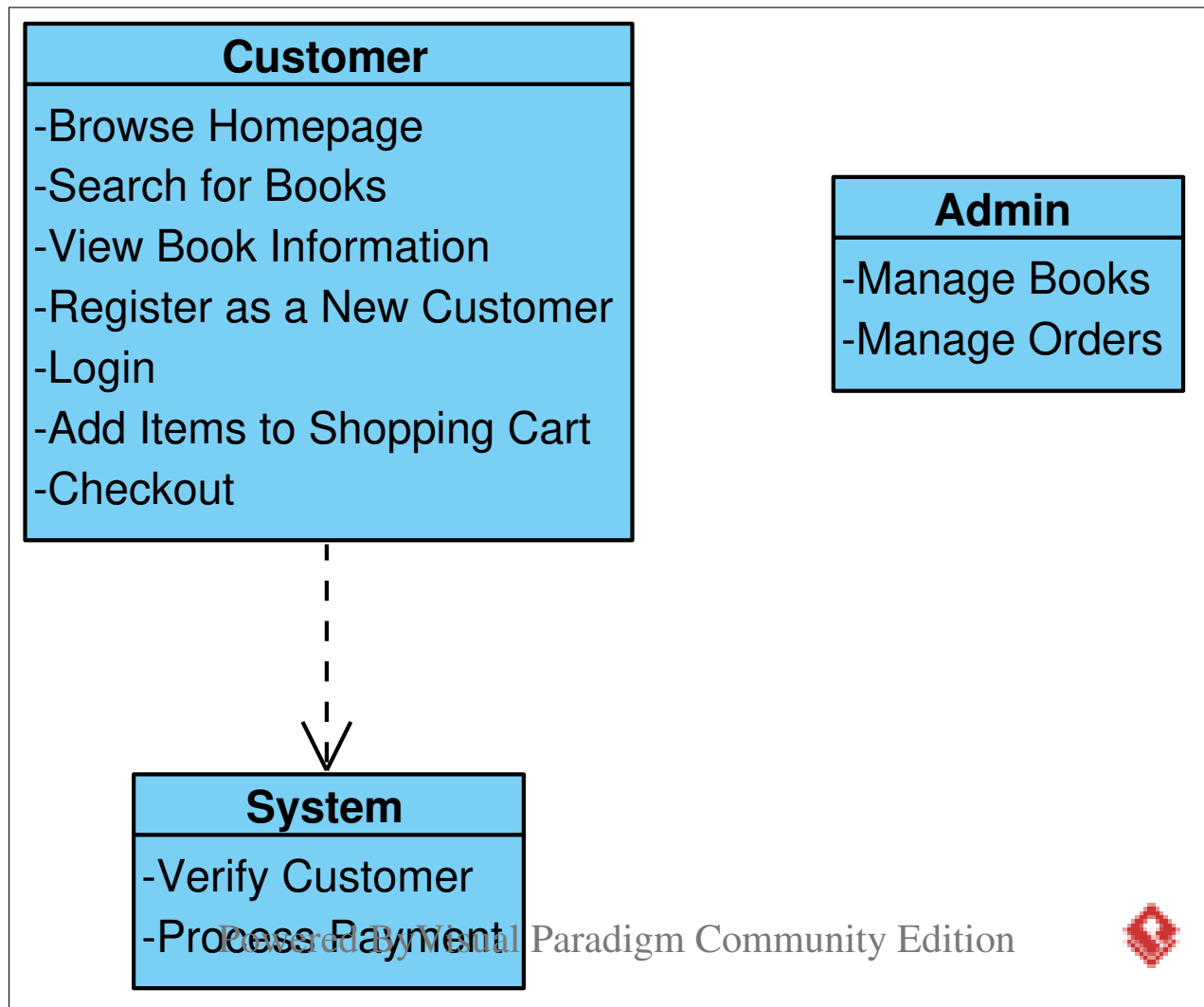


Figure 1.9: Use-case Diagram for E-Book Website

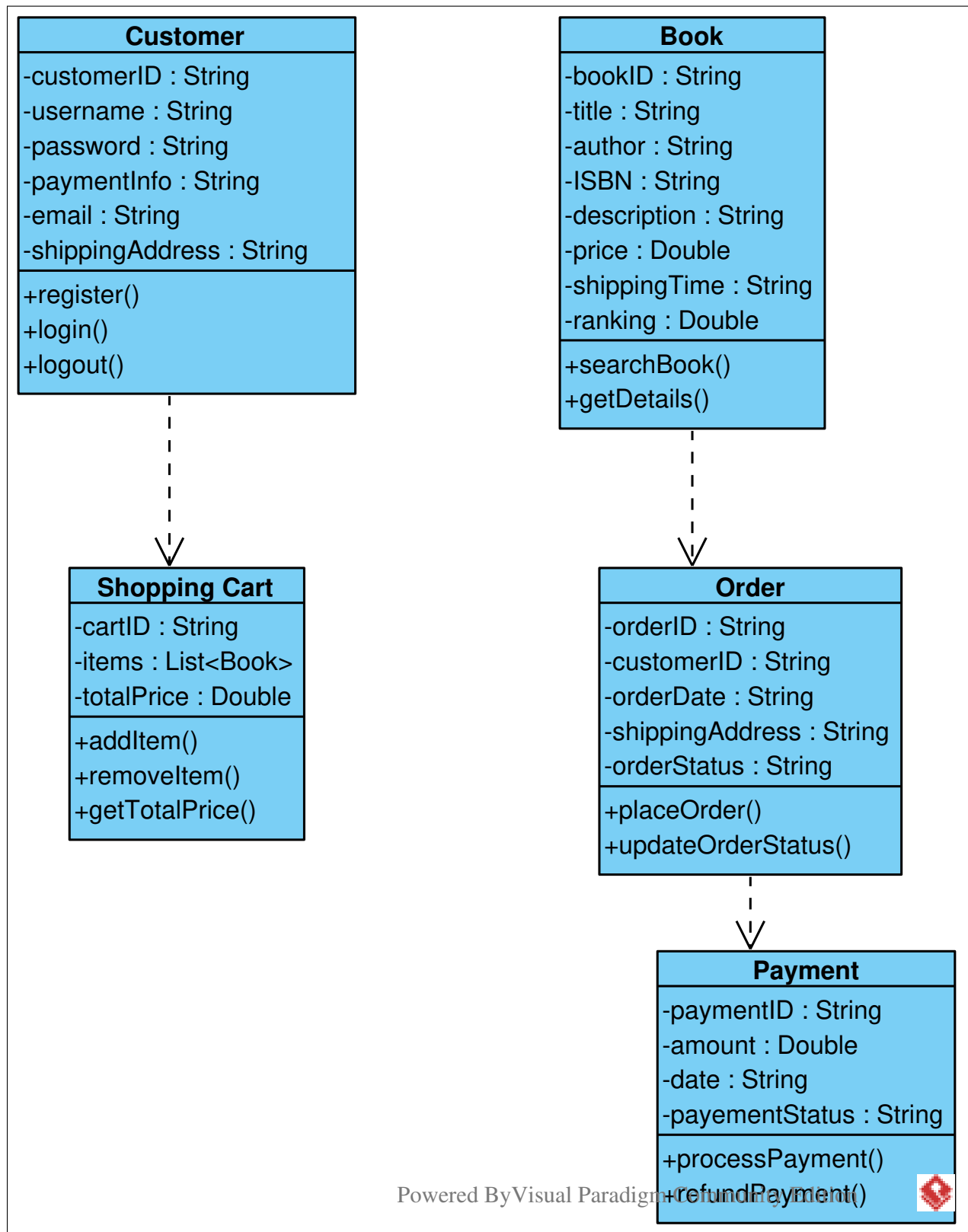


Figure 1.10: Class Diagram for E-Book Website

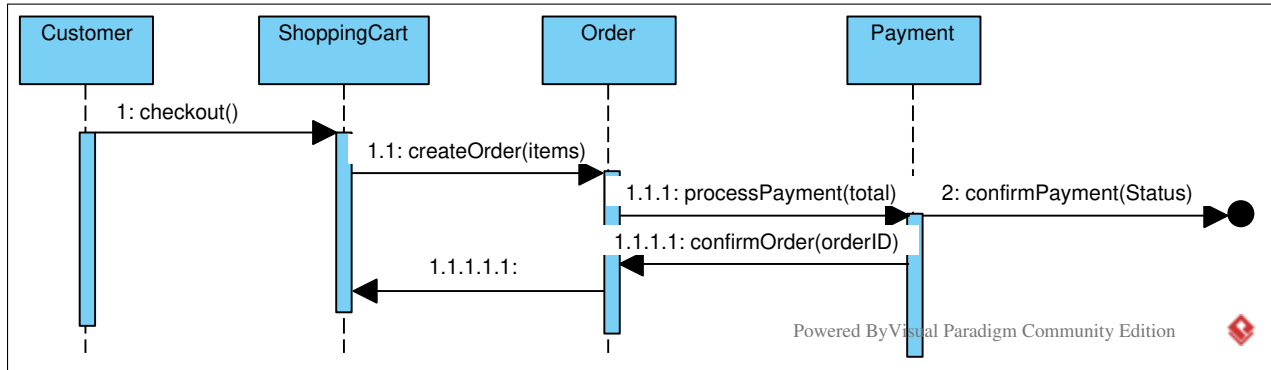


Figure 1.11: Sequence Diagram for E-Book Website

Chapter 2

Manual Simulink Lab

– Kyle Ponte, Enoch Chan

2.1 Exercise 7.1

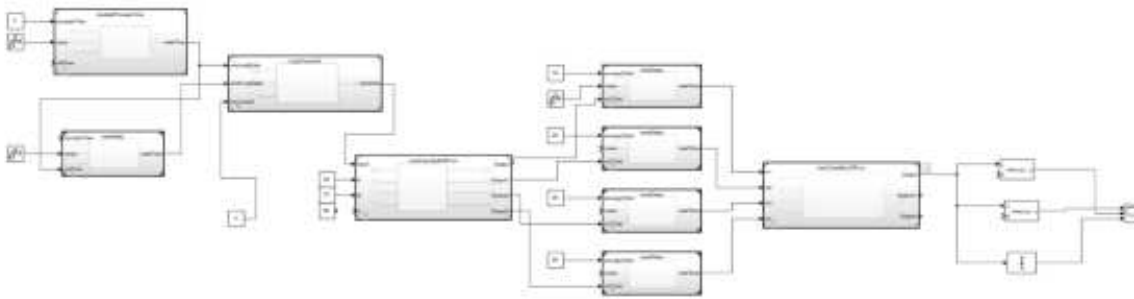


Figure 2.1: Diagram of Exercise 7.1 Model

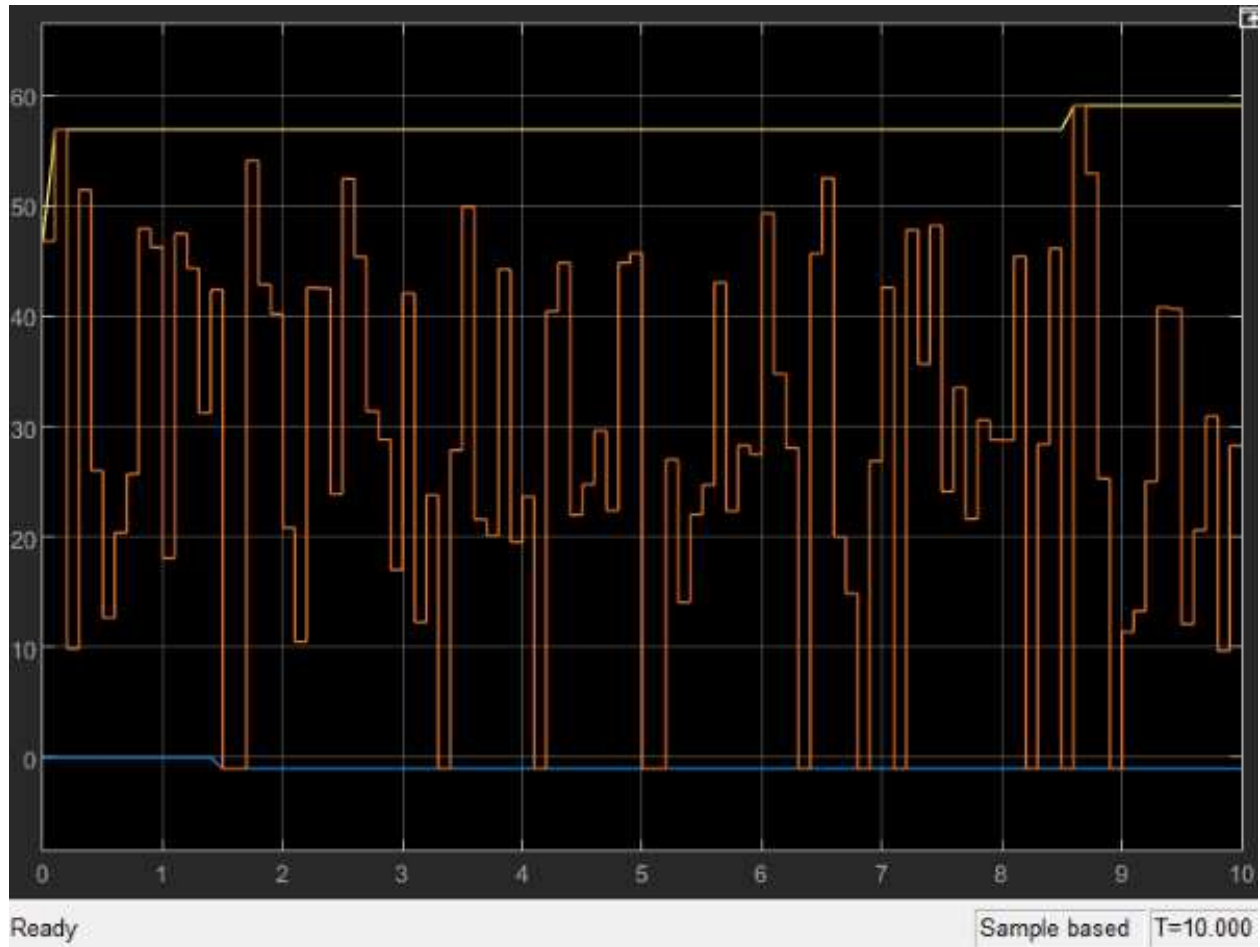


Figure 2.2: Simulink Scope Showing Min, Max, and Average

2.2 Exercise 7.2

Manual Calculations:

$$1 + 2 + 1 + 1 + 1 = 6$$

$$1 + 2 + 1 + 1 + 2 = 7$$

$$1 + 2 + 1 + 2 + 1 = 7$$

$$1 + 2 + 1 + 2 + 2 = 8$$

$$1 + 2 + 2 + 2 + 1 = 8$$

$$1 + 2 + 2 + 2 + 2 = 9$$

$$1 + 2 + 2 + 2 + 1 = 8$$

$$1 + 2 + 2 + 2 + 2 = 9$$

$$1 + 3 + 1 + 1 + 1 = 7$$

$$1 + 3 + 1 + 1 + 2 = 8$$

$$1 + 3 + 1 + 2 + 1 = 8$$

$$1 + 3 + 1 + 2 + 2 = 9$$

$$1 + 3 + 2 + 2 + 1 = 9$$

$$1 + 3 + 2 + 2 + 2 = 10$$

$$1 + 3 + 2 + 1 + 1 = 8$$

$$1 + 3 + 2 + 1 + 2 = 9$$

Search Times:

Fastest:

$$1 + 2 + 1 + 1 + 1 = 6$$

Slowest:

$$1 + 3 + 2 + 2 + 2 = 10$$

Average:

$$\text{Sum} = 6 + 7 + 7 + 8 + 8 + 9 + 8 + 9 + 7 + 8 + 8 + 9 + 9 + 10 + 8 + 9 = 130$$

$$\text{Average} = \frac{130}{16} = 8.125$$

These calculations represent the total execution times for different search paths in a binary tree, with each number corresponding to the time taken at each level of the tree. The fastest search time is $6\mu s$, occurring when the search reaches a leaf node through a short path with minimal branching. The slowest search time is $10\mu s$, occurring when the search takes the longest path through the tree. The average search time is calculated by summing all the path times and dividing by the total number of paths, resulting in an average of $8.125\mu s$, which reflects the typical execution time across all possible search paths.

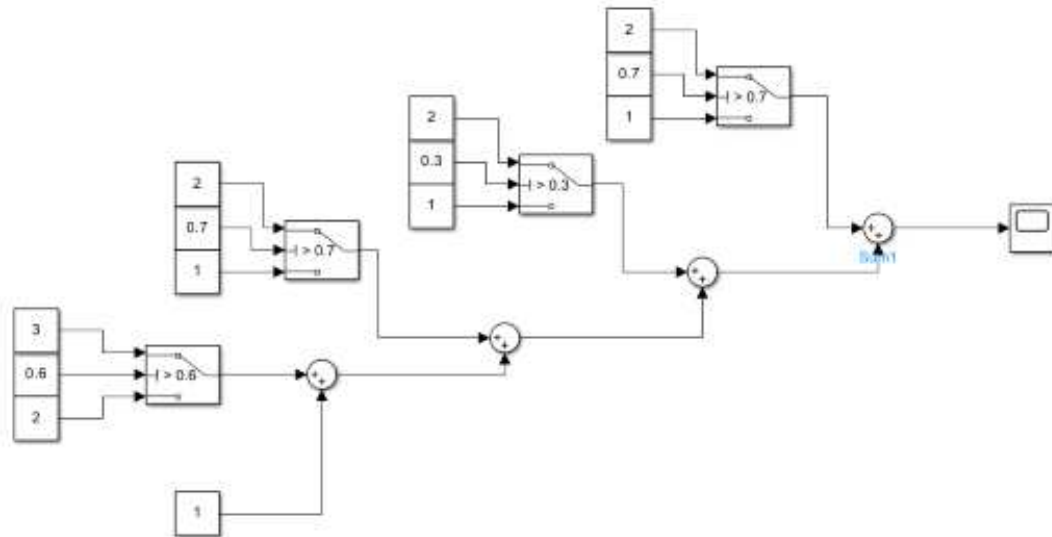


Figure 2.3: Fastest Time

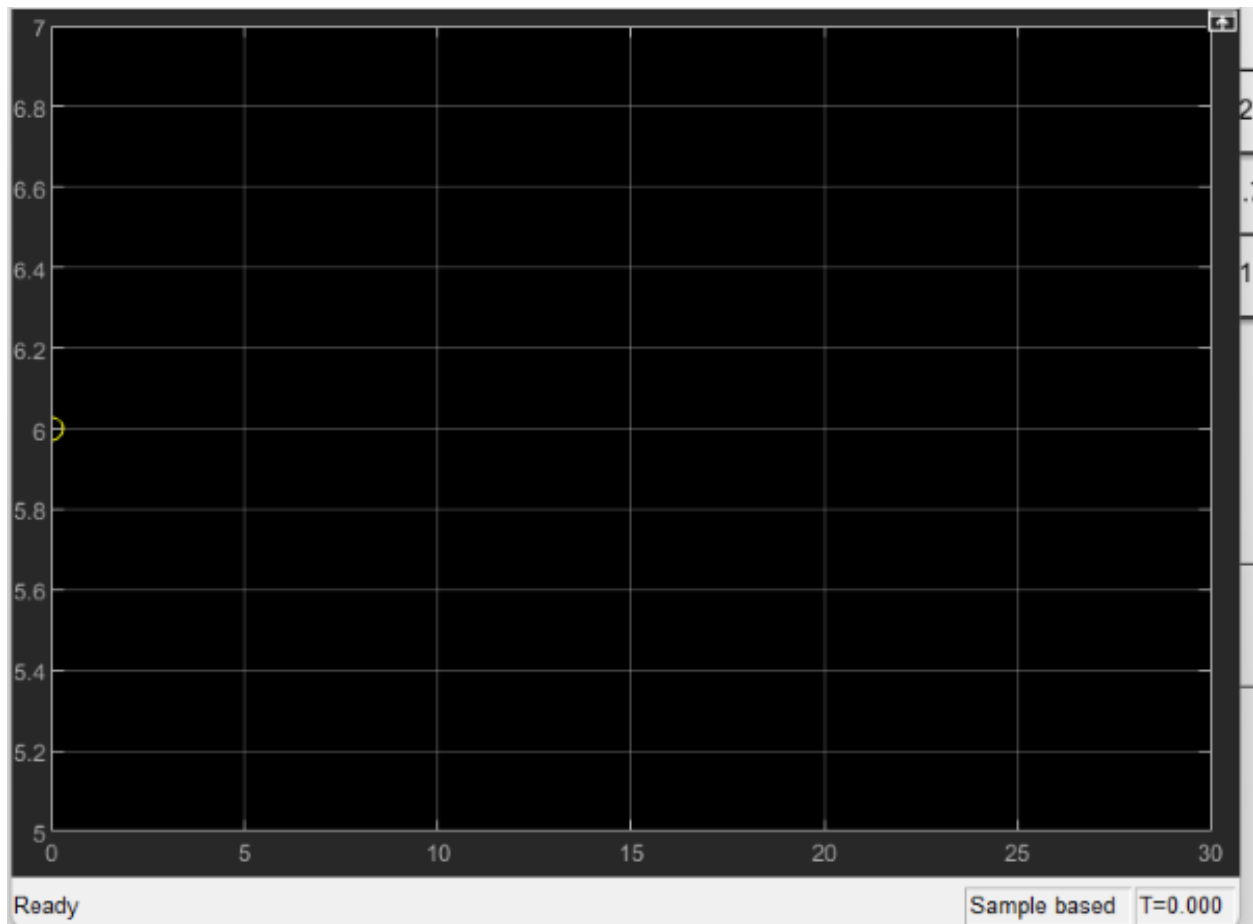


Figure 2.4: Fastest Time

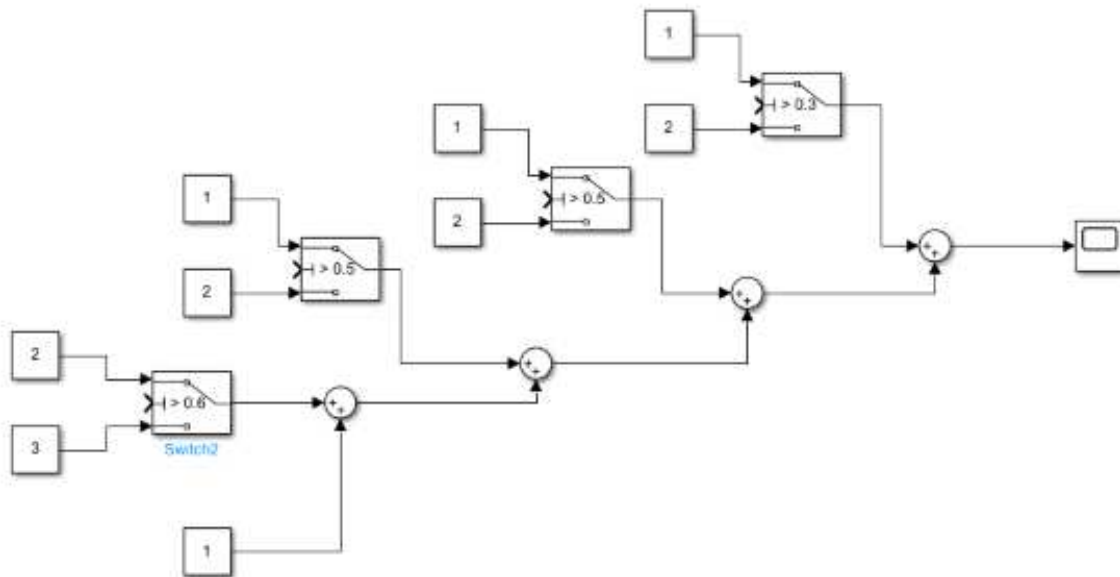


Figure 2.5: Fastest Time

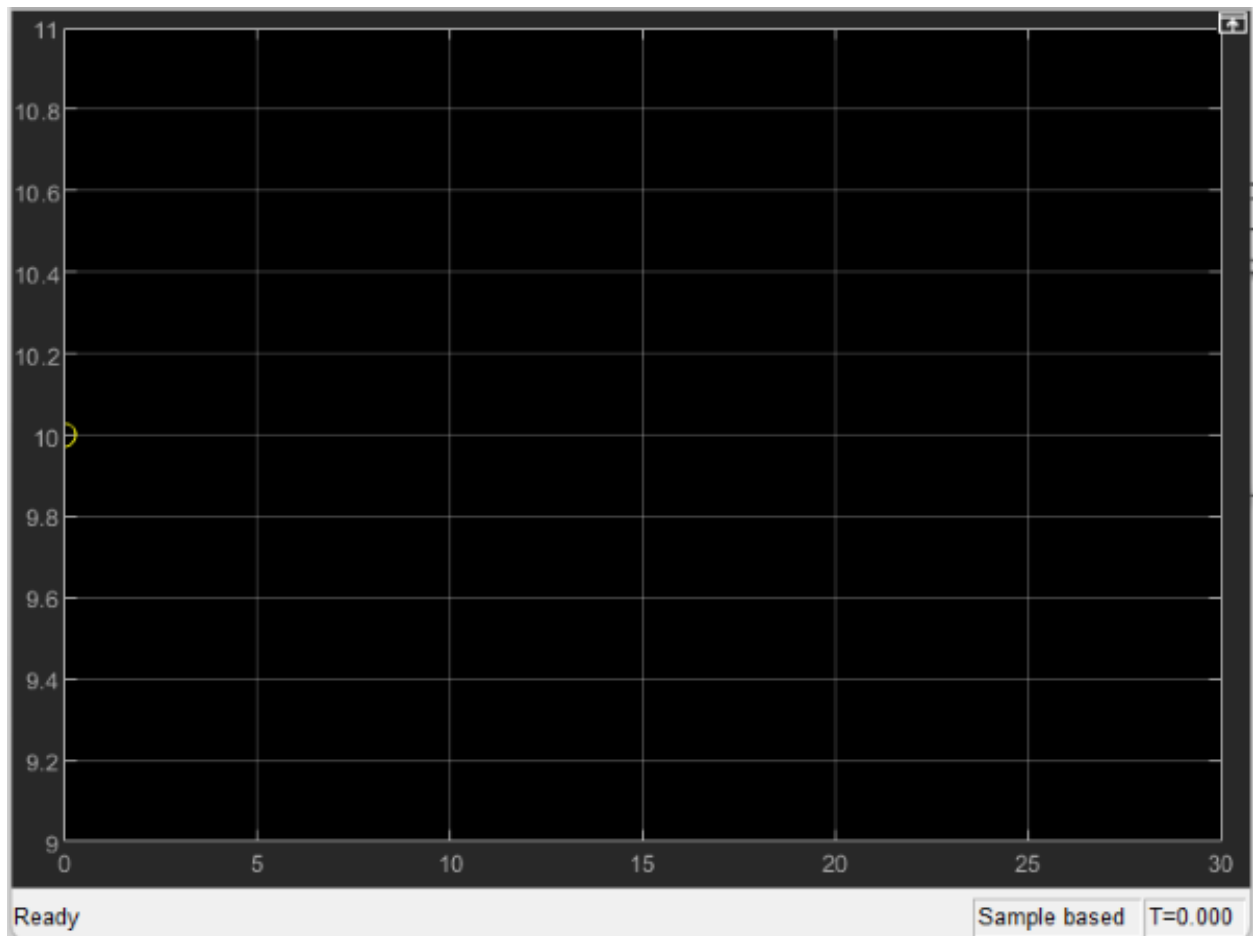


Figure 2.6: Fastest Time